BitLocker for Windows Vista

- Once the O.S. was installed, it was too complicated to set up
- Users had to shrink the system partition to make space for the BitLocker partition
- Microsoft addressed this by releasing BitLocker Drive Preparation Tool (BDPT) [2]

BitLocker for Windows 7

- BDPT is <u>integrated</u> in Windows 7 and greatly simplifies the encryption of a system drive, just turn it on and the BDPT does the rest!
- Protection is extended to cover removable drives with BitLocker To Go [3]

Comparison between BitLocker for Vista and Windows 7

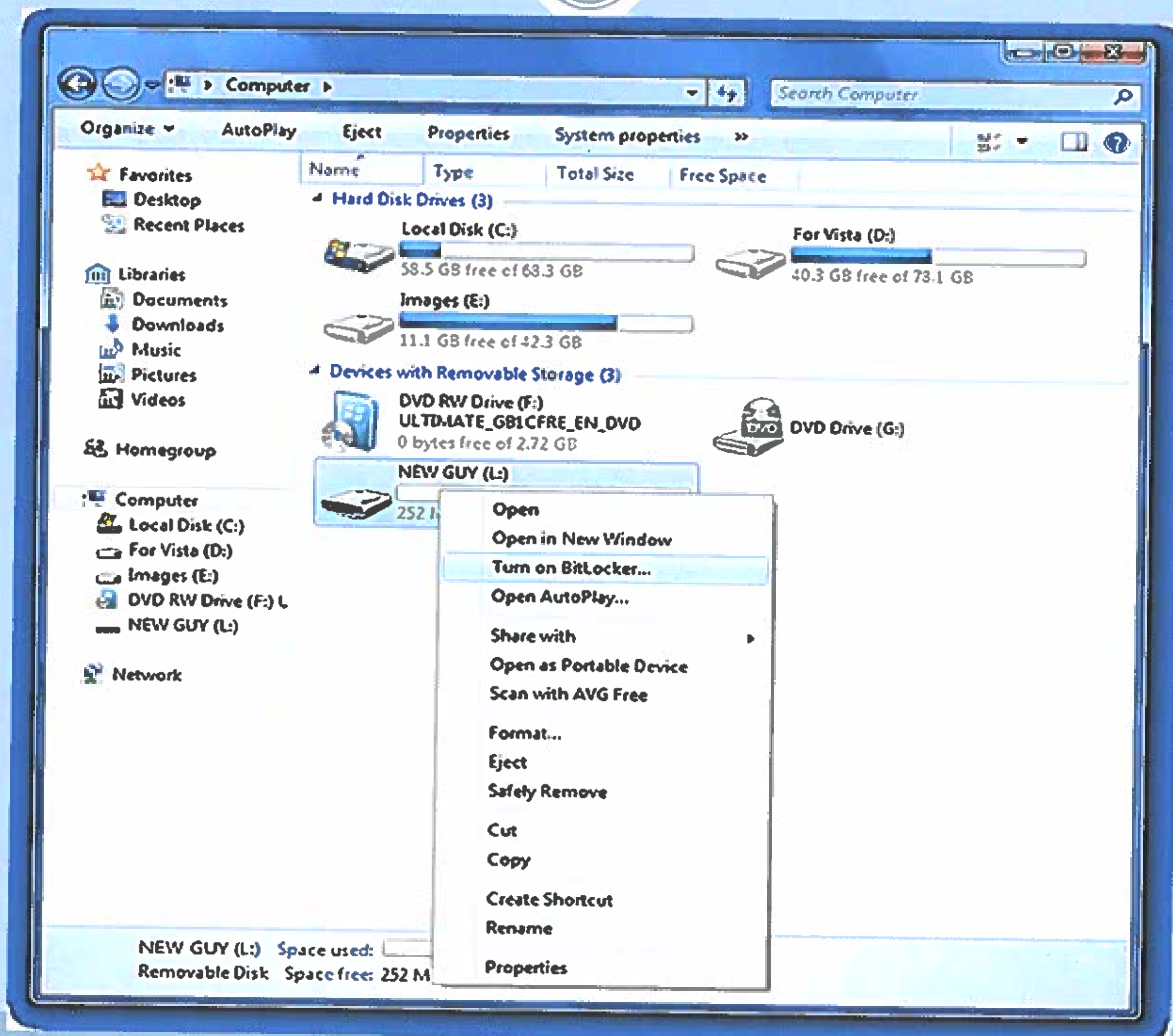
Windows Vista

- Only for Ultimate/Enterprise Editions
- Has a drive letter and is not hidden
- Too complicated to set up
- Supports Active Directory integration

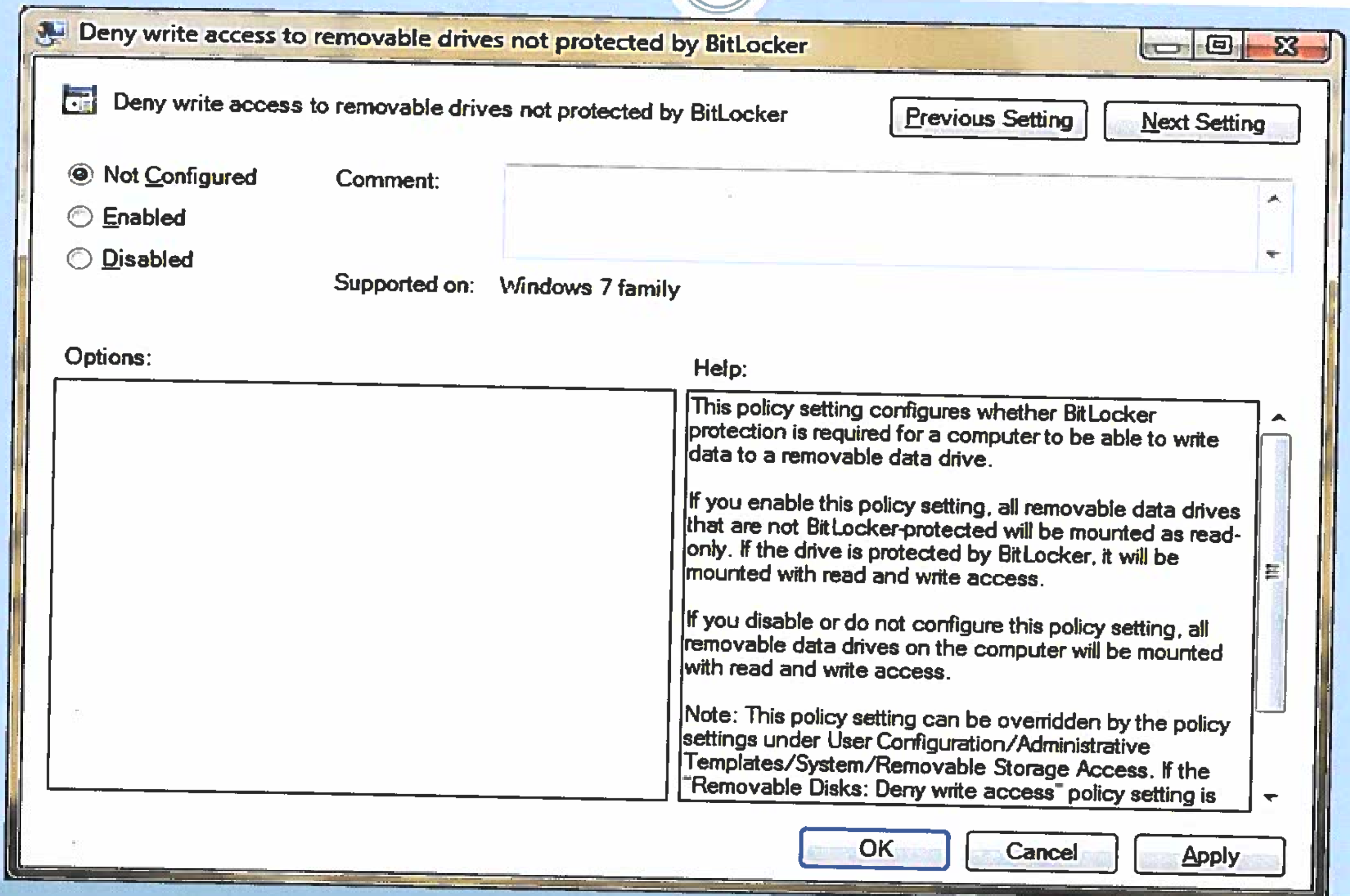
Windows 7

- Only for Ultimate/Enterprise Editions
- Has no drive letter and is hidden
- Easier to set up
- Supports Data Recovery Agent

Screenshots of Windows 7 using BitLocker



Screenshots of Windows 7 using BitLocker (Cont)



Sources

- [1] Wikipedia, *BitLocker Drive Encryption*, 29 Nov 2009, available from http://en.wikipedia.org/wiki/Bitlocker; Internet; accessed 01 Dec 2009.
- [2] 4Sysops, *Review: Windows 7 BitLocker*, 2009, available from http://4sysops.com/archives/review-windows-7-bitlocker/; Internet; accessed 01 Dec 2009.
- [3] PCmag.com, *Security in Windows 7: BitLocker and More*, 21 Nov 2009, available from http://www.pcmag.com/article2/0,2817,2335346,00.asp; Internet; accessed 01 Dec 2009.

eCryptfs: A cryptographic filesystem in the Linux kernel using stacking technology

Josiah Vergonio
COINS 431
Advanced Operating Systems

Introduction

- Based off of what we learned in class about security- security is only as strong as the weakest link (humans)
- Data Encryption needs to be made <u>transparent</u>, flexible, easily deployed while still maintaining security strong enough to counter sophisticated attacks
- The operating system itself should provide a system-wide data encryption service for sensitive information written to secondary storage (CDs, USB drives, etc.)

Popular Cryptographic Filesystem Solutions

- Dm-crypt: comes with Linux kernel and provides block device layer encryption (block devices are devices through which the system moves data in blocks, typically, hard disks, CD-ROM drives, or memory-regions)
- Loop-AES, TrueCrypt are others separately obtained from official Linux kernel

Advantages of block device layer encryption

- Simple in concept and implementation
- Attackers learn **nothing** about the filesystem without the key
- Sparse files can be supported in filesystems on encrypted block devices

Disadvantages of block device layer encryption

- Fixed region of storage must be pre-allocated for the entire filesystem
- It can be difficult to change encryption keys or ciphers
- No flexibility for the block device encryption mechanism to encrypt different files with different keys or ciphers
- All content in filesystem incurs overhead of encryption/decryption, including data that does not require secrecy

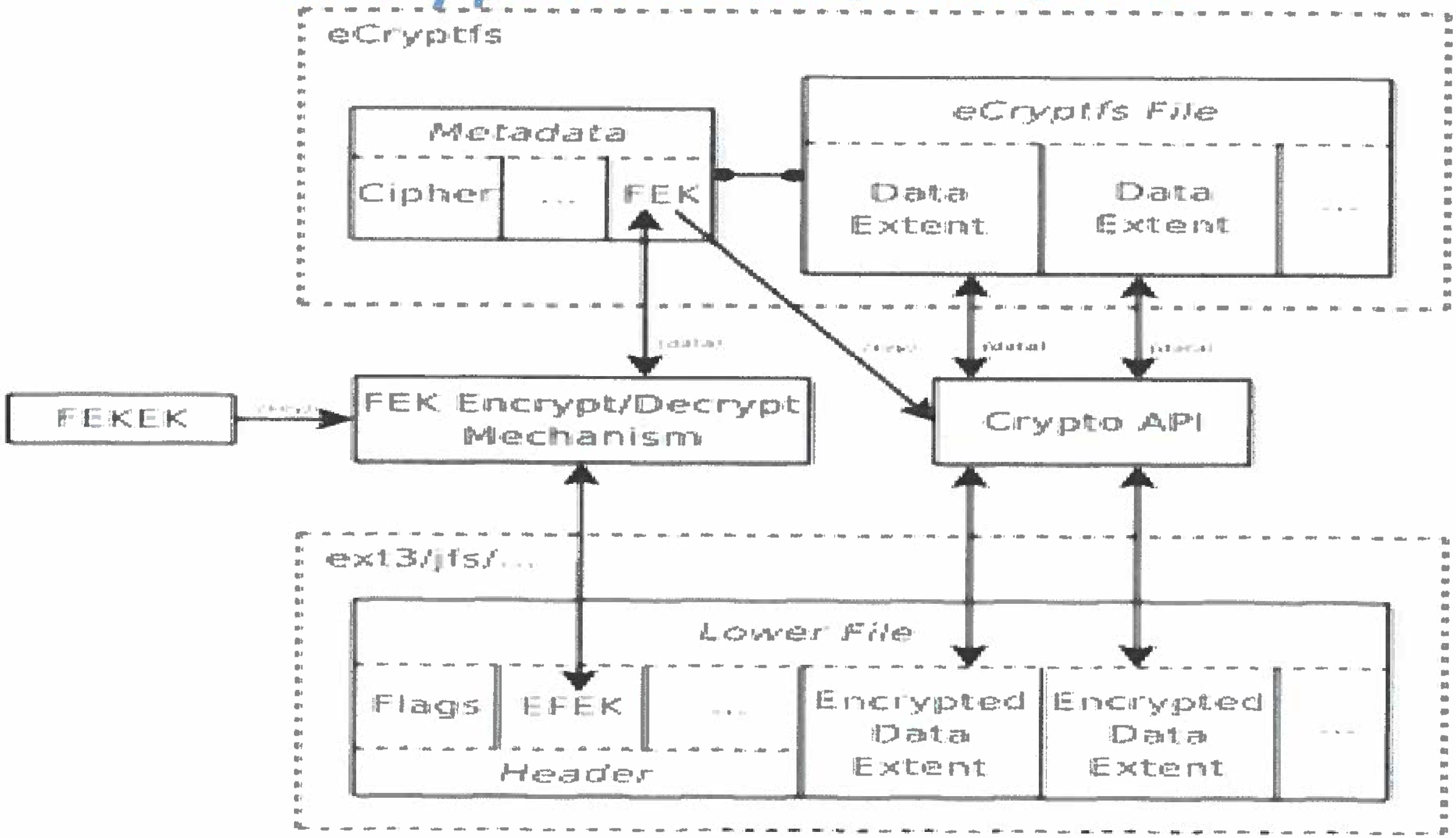
EncFS

- A user-space cryptographic filesystem for Linux that operates via FUSE (Filesystem in Userspace)
- User-space filesystems are easier to implement than kernel-native filesystems
- Unlike, block device encryption, EncFS operates as an actual filesystem
- EncFS encrypts/decrypts individual files
- <u>Disadvantages</u>: Performance overhead from frequent kernel/user-space context switches and current lack of support for shared writeable memory mappings

eCryptfs

- Kernel-native stacked cryptographic filesystem for Linux
- Stacked filesystems layer on top of existing mounted filesystems that are referred to as lower filesystems
- eCryptfs encrypts/decrypts files as they are written to or read from the lower filesystem
- Apps in user space make system calls with kernel VFS
- eCryptfs and lower filesystem (ext3, JFS, NFS, etc) are registered in kernel VFS

eCryptfs Illustration 2



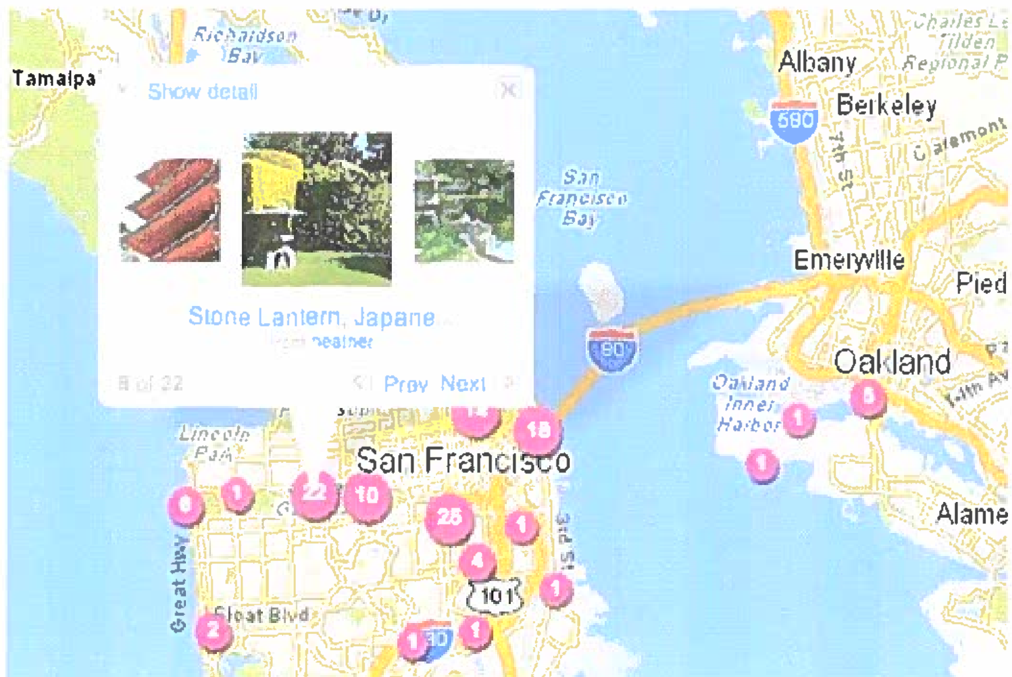
Conclusion

- eCryptfs is an effort to reduce the barriers that stand in the way of effective and universal use of file encryption
- eCryptfs provides the means whereby a cryptographic filesystem solution can be more easily and effectively deployed transparently

Creating a Geotagging Application on the Palm Pre

The Goal of this project

- To take a picture with the Palm Pre and know where that picture was taken at by retrieving the GPS coordinates from the phone at a later date.



Algorithm

1. Have application access the camera.
2. When a picture is taken, get a current position GPS fix and store this data.
3. Access the data later on.
4. Process the data into a location that the user can understand.

What I Have Learned So Far

- WebOS
  - The Operating System of the Palm Pre
  - Optimized for launching and managing multiple applications at once
  - Supports JavaScript, CSS, and HTML
- Stages and Scenes
  - A *stage* is a card
  - A *scene* is a view of the app within a stage
  - In class visual aid

What I Have Learned So Far (Cont.)

- How will we access the camera/location services on the Palm Pre?
  - Use the services API*; it is used to access hardware features (location, the phone, the camera)
  - Called through a single function: Mojo.Service.Request()

***A**pplication **P**rogramming **I**nterface: defines the ways by which an application may request services for libraries and/or operating systems

What I Have Learned So Far (Cont.)

- How will we store the GPS data so we can access it later on?
  - Current research suggests that the Pre can utilize four different types of data storage
    - Cookies
    - Depots
    - HTML 5 storage
    - Ajax

Possible Problems

- Lack of experience creating apps for Palm Pre might cause programming frustration
  - Possible solutions: ask help from online developer forums, ask Dr. Lin for help, various reference documentation online and Palm WebOS eBook

Possible Problems (cont)

- Finding time to work on senior project despite being a full time student and going to work
  - Possible solutions: have a timeline to stay on track, practice good time management skills

Tentative Timeline

- Presentation/Install SDK*-- today
- Research -- 3 weeks
- Set up programming environment -- 3 weeks
- Write code/Debug code-- 3 weeks
- Get feedback -- 1 week
- Implement changes (if needed) – 3 weeks

*Software Development Kit: allow software engineers to create apps for certain software packages, operating systems, computer system or similar platform

Picture of Emulator